



CS670: Cryptographic Techniques for Privacy Preservation

Module 3: Zero Knowledge Proofs

Adithya Vadapalli



*A **zero-knowledge proof** (also known as a **ZK proof** or **ZKP**) is a protocol in which one party (the prover) can convince another party (the verifier) that some given statement is true, without conveying to the verifier any information beyond the mere fact of that statement's truth.*

Zero Knowledge Proofs



Verifier



Prover



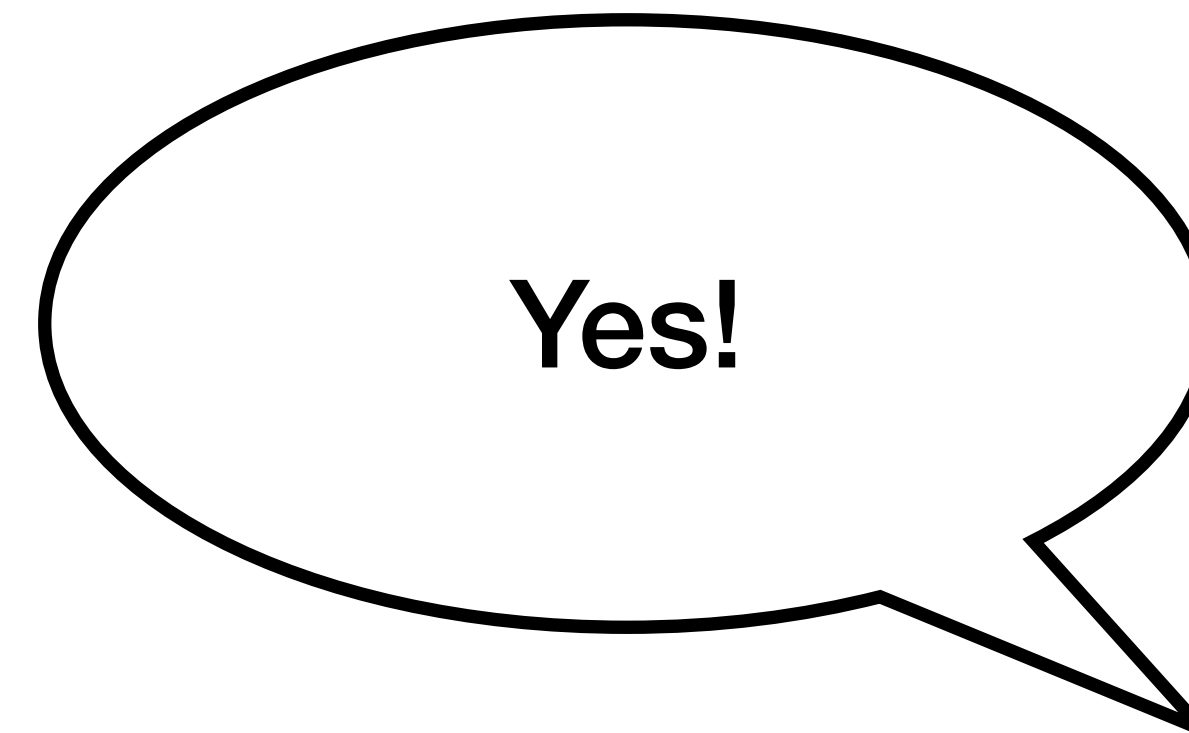
Prover wants to distinguish he can distinguish between two colors

Zero Knowledge Proofs



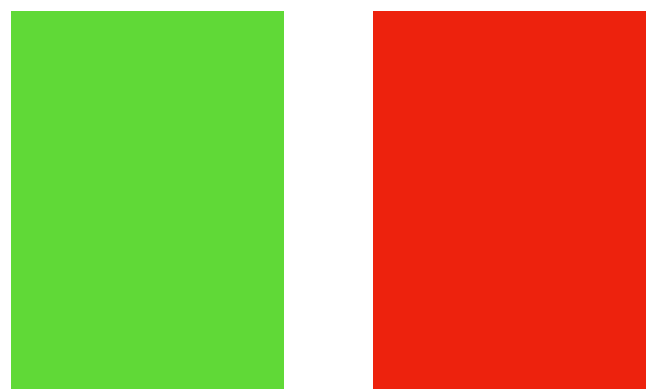
Verifier

Did I swap the cards?



Prover

Yes!



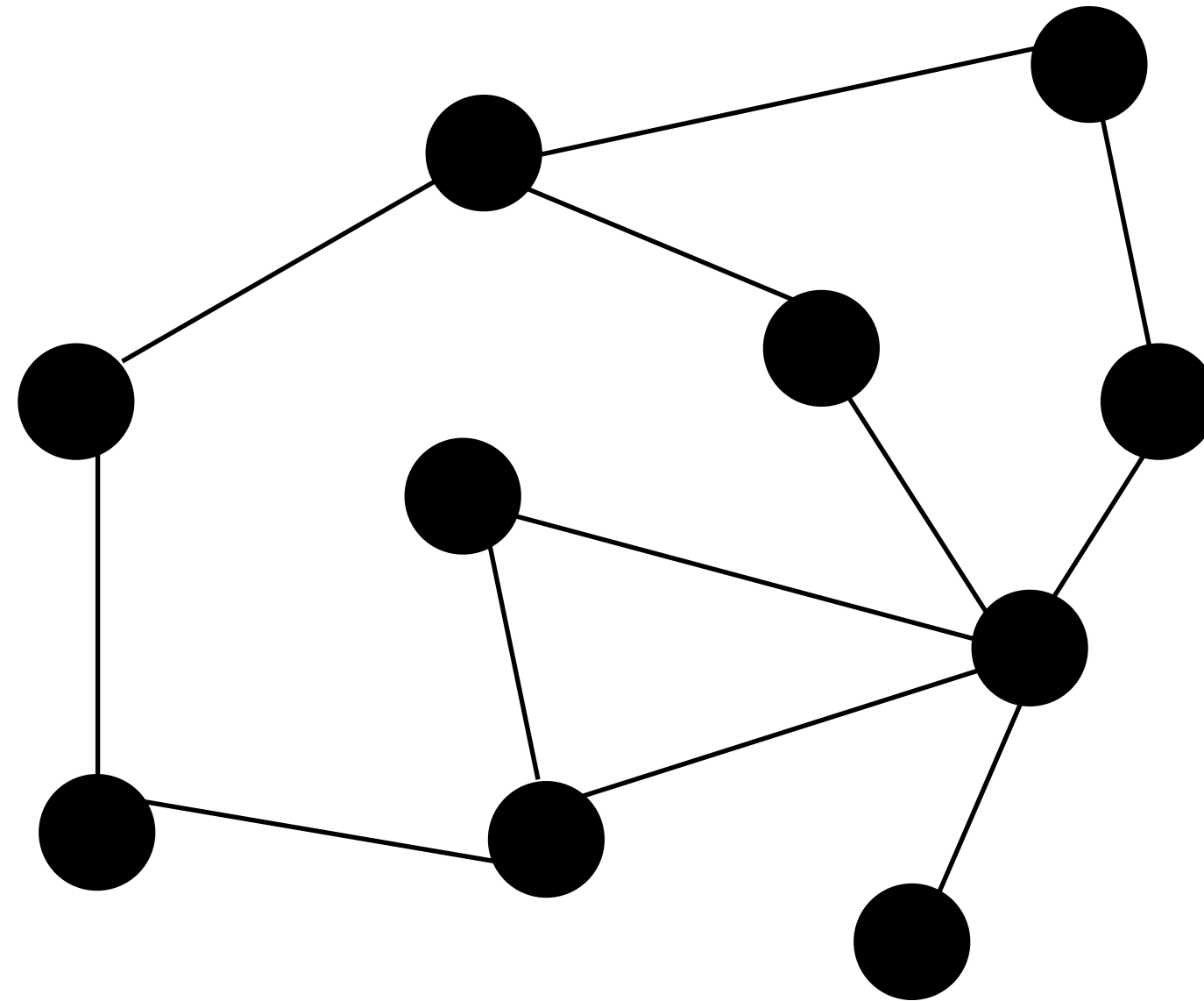
If the prover answers correctly, the verifier accepts the proof

What is the probability with which a prover can cheat? $\frac{1}{2}$

Zero Knowledge Proofs



Verifier



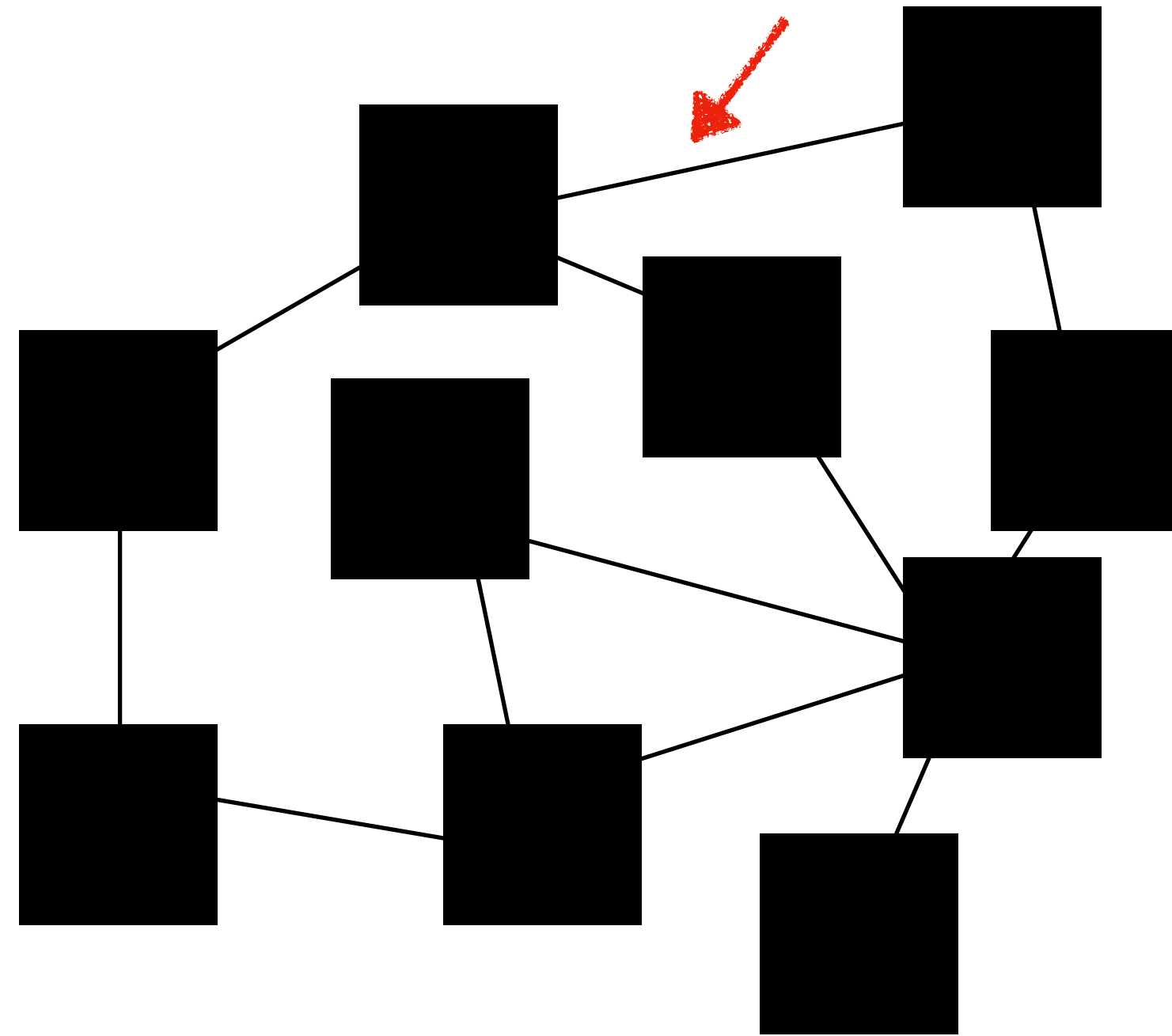
Prover

Prover would like to prove that they *know* a 3-coloring of the graph

Zero Knowledge Proofs



Verifier



Prover

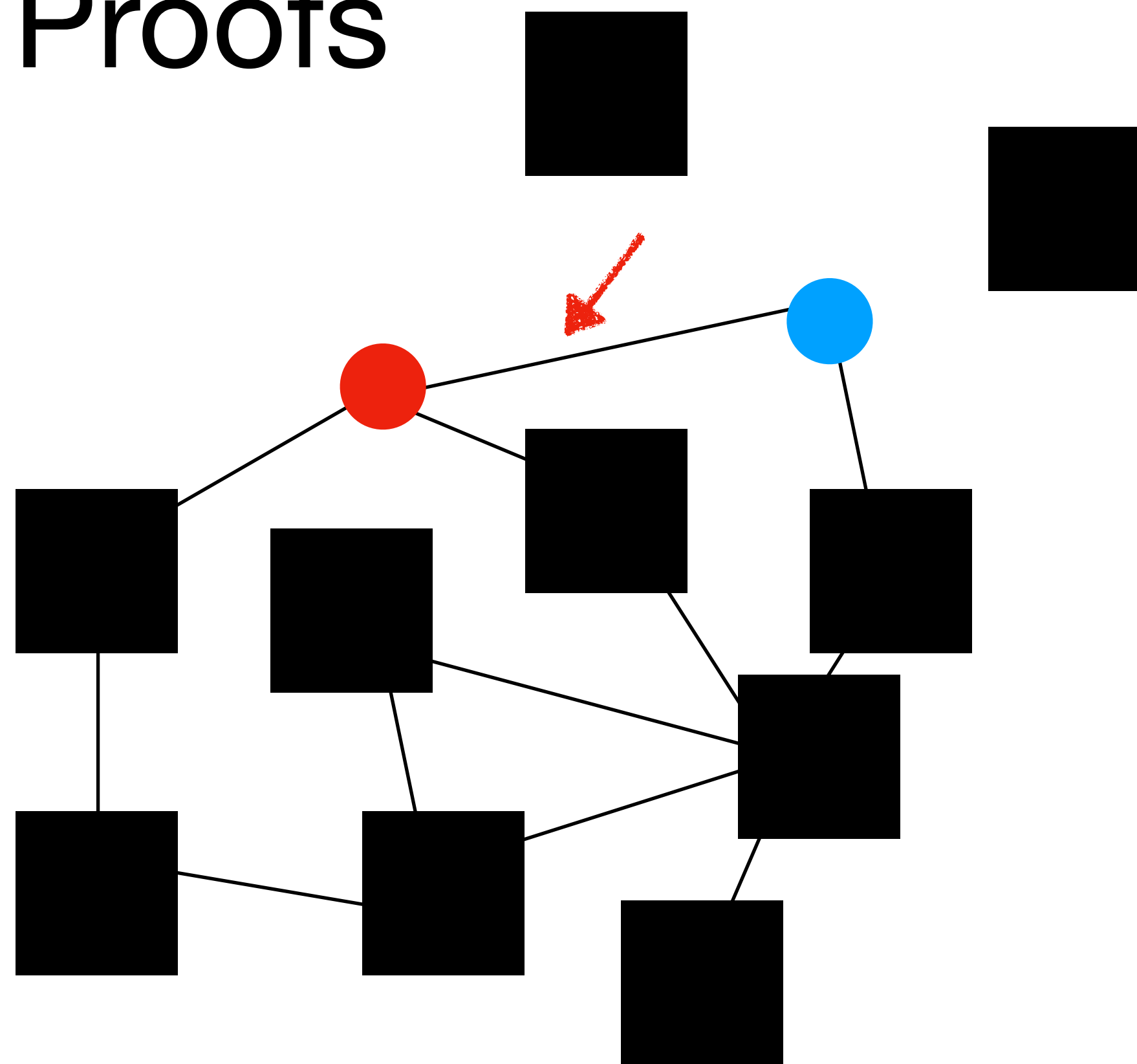
Prover colors the graph and covers them!

Verifier picks one random edge of her choice.

Zero Knowledge Proofs



Verifier



Prover

Prover colors the graph and covers them!

Verifier picks one random edge of her choice.

Prover opens that edge — and verifier accepts if they are colored differently

Midsem Grading is Complete (Finally!)

Marks will be published by the EOD after we give some comments and double check some questions

Current Stats

Max: 232/250

Median: 175.5

Mean: 156.19

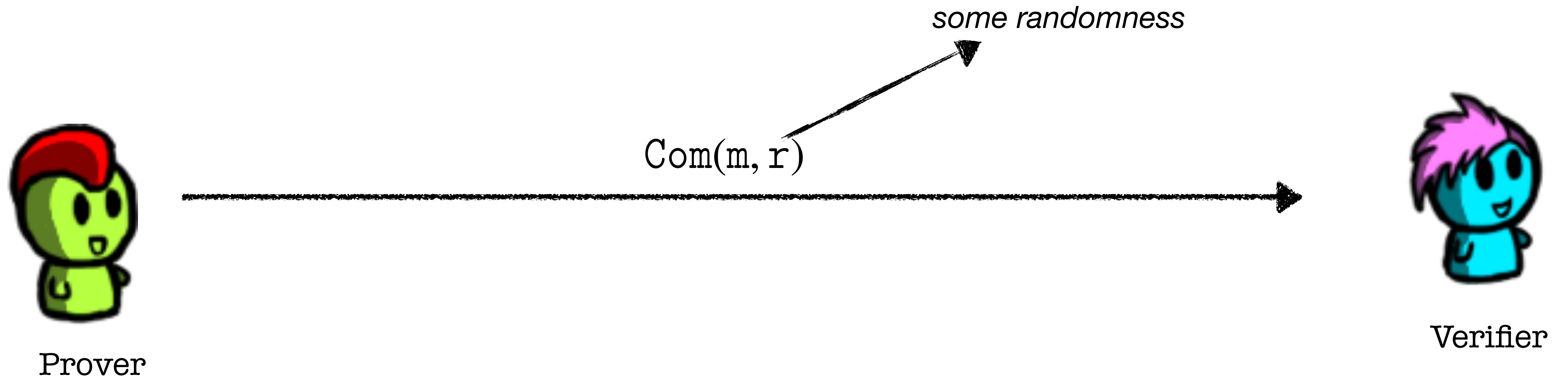
Assignment 2 is out!

Assignment 1 marks will be given to you by the end of Monday

Quiz 1 regrade requests (there are not many) will be taken care of this week

Quiz 2 marks will be out end of next week.

Commitment Schemes



Computationally Hiding: $\text{Com}(m_0, r)$ $\text{Com}(m_1, r)$ are computationally indistinguishable

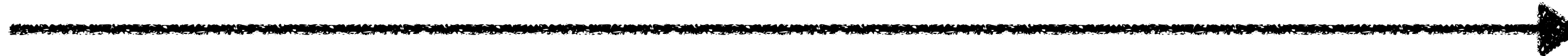
Computationally Binding: Cannot find (m, r) (m', r') such that: $\text{Com}(m, r) = \text{Com}(m', r')$

Sigma Protocols

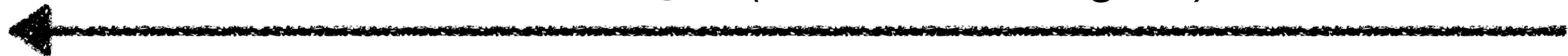


Prover

U (*generate a commitment U*)



C (*send a challenge C*)



Z (*send a challenge Z*)



Verifier

Sigma Protocols for Discrete Log

$$h = g^w$$



Prover

h , g are public



Verifier

Prover wants to prove that she knows a w , $h = g^w$

Sigma Protocols: An Example



Prover

Let p be a prime

Let q be a prime divisor in $(p-1)$

g is an element of order q in Z_p^*

Suppose the prover has chosen $w \in Z_q$

Suppose prover publishes: $h = g^w \bmod p$

Verifier receives: (p, q, g, h)



Verifier

Verfier Checks:

p and q are prime

g has order q

h has order dividing q

Since there is only one subgroup of order $q \in Z_p^*$
(once g is fixed)

Therefore, if h has order dividing q

then, it must lie in: $\langle g \rangle = \{g^0, g, g^2, \dots, g^{q-1}\}$

Sigma Protocols: An Example

How does the verifier check: g has order q ?

$$g^q \equiv 1 \pmod{p}$$

How does the verifier check: h has order dividing q ?

$$h^q \equiv 1 \pmod{p}$$

Sigma Protocols: An Example

Verfier Checks:

p and q are prime

g has order q

h has order dividing q

$$\langle g \rangle = \{g^0, g, g^2, \dots, g^{q-1}\}$$

Since there is only one subgroup of order q $q \in \mathbb{Z}_p^*$ once g is fixed

Therefore, if h has order dividing q then, it must lie in $\langle g \rangle$

Does this prove that Prover knows w?

Schnorr Protocols



Prover

$$r \in \mathbb{Z}_q: \quad a = g^r$$

$$e \in \mathbb{Z}_{2^t}$$

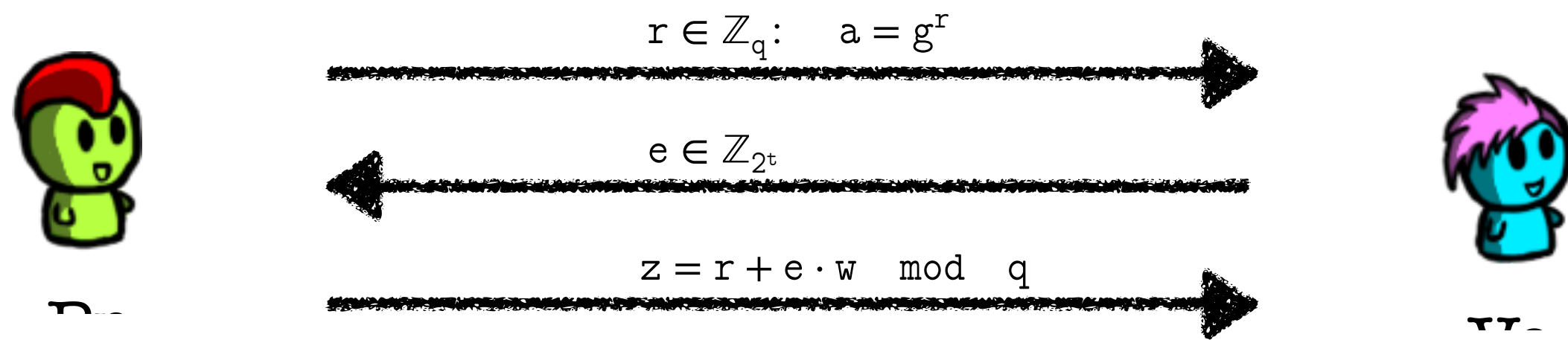
$$z = r + e \cdot w \mod q$$



Verifier

checks: $g^z = ah^e \mod p$

Schnorr Protocols



Suppose that there are two different challenges e and e' to which the prover can find a response.

$$g^z = ah^e$$

$$g^{z'} = ah^{e'}$$

$$g^{z-z'} = h^{e-e'} \quad h = g^{(z-z') \cdot (e-e')^{-1}}$$

Therefore, a cheating prover can only answer one of the challenges!

Schnorr Protocol Properties

Completeness

If a prover knows the witness w : he should always be able to prove it

Special Soundness

(Informal version: If the prover does not know the witness w , he should not be able to prove it!)

There exists an efficient extractor A that given any x and a pair of transcripts

(a, e, z) (a, e', z') *with* $e \neq e'$

$(x, w) \in R$

Schnorr Protocol Properties

Special Soundness

(Informal version: If the prover does not know the witness w , he should not be able to prove it!)

What is the probability with which a prover can cheat in the previous protocol?

$$1/2^{-t}$$

Schnorr Protocol Properties

Special Honest Verifier ZK:

an algorithm

There exists a *simulator* S that, given any x and e , outputs an accepting transcript (a, e, z) that distributes exactly like a real execution where V sends e .

Sigma Protocols

Prover sends a message **a**

Verifier sends a random *t-bit* string **e**

Prover sends a reply **Z**

Verifier decides whether to accept or reject based on the data that they have seen:

(x, a, e, z)

Sigma Protocols

3-Move Protocol, and we have completeness:

If prover and verifier follow the protocol on input x and private input w $(x, w) \in R$ then verifier always accepts

Special Soundness Property:

(a, e, z) (a, e', z') $e \neq e'$ Can efficiently compute $(x, w) \in R$

There exists a polynomial-time simulator S , which on input x and a random e outputs an accepting conversation of the form (a, e, z) with the same probability distribution as conversations between the honest P, V on input x .

Proof of Knowledge

Defining *proof of knowledge* is not straightforward.

Key challenge: What does it mean for the prover to know something?

Does the information have to appear in the memory explicitly?

A machine knows something if it can efficiently compute it.

Intuition: If something can be extracted from the Prover, it must have it!

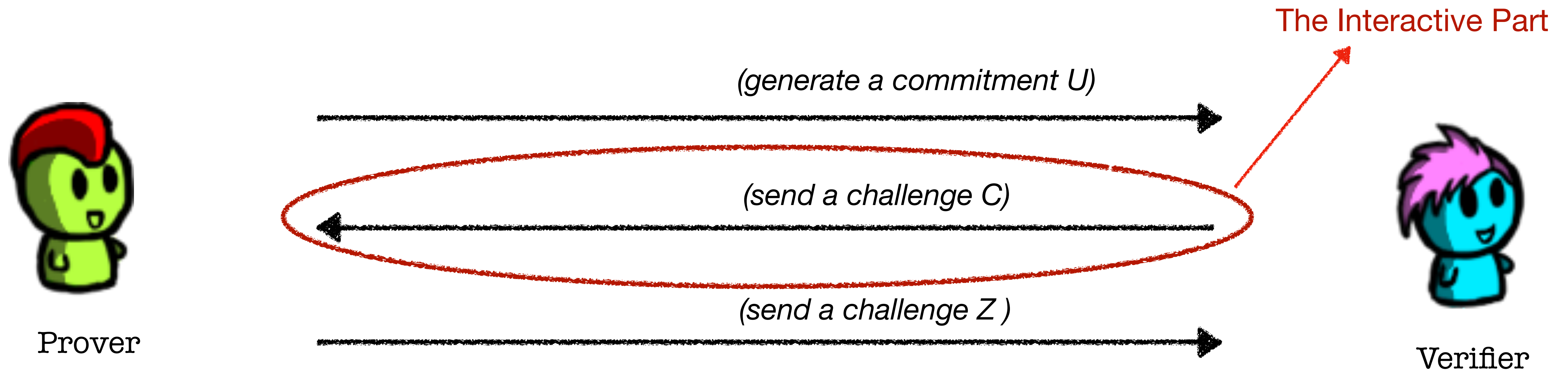
Honest Verifier Zero Knowledge

Why is this an honest verifier zero-knowledge proof?

The verifier can send something other than a random e

Fiat Shamir Transformation

Can we make the Zero Knowledge Proof Non-Interactive?



Fiat Shamir Transformation

Can we make the Zero Knowledge Proof Non-Interactive?

To prove a statement ~~x~~ *non-interactively*



Prover

Generate: a

Compute: $e = H(a, x)$

Compute: z



Verifier

$a \ e \ z$



OR Proof

Given two inputs: x_0, x_1 Prover knows w

$(x_0, w) \in R$ or $(x_1, w) \in R$

The prover wants to convince the verifier that they know a w such that at least one of the above conditions is true.

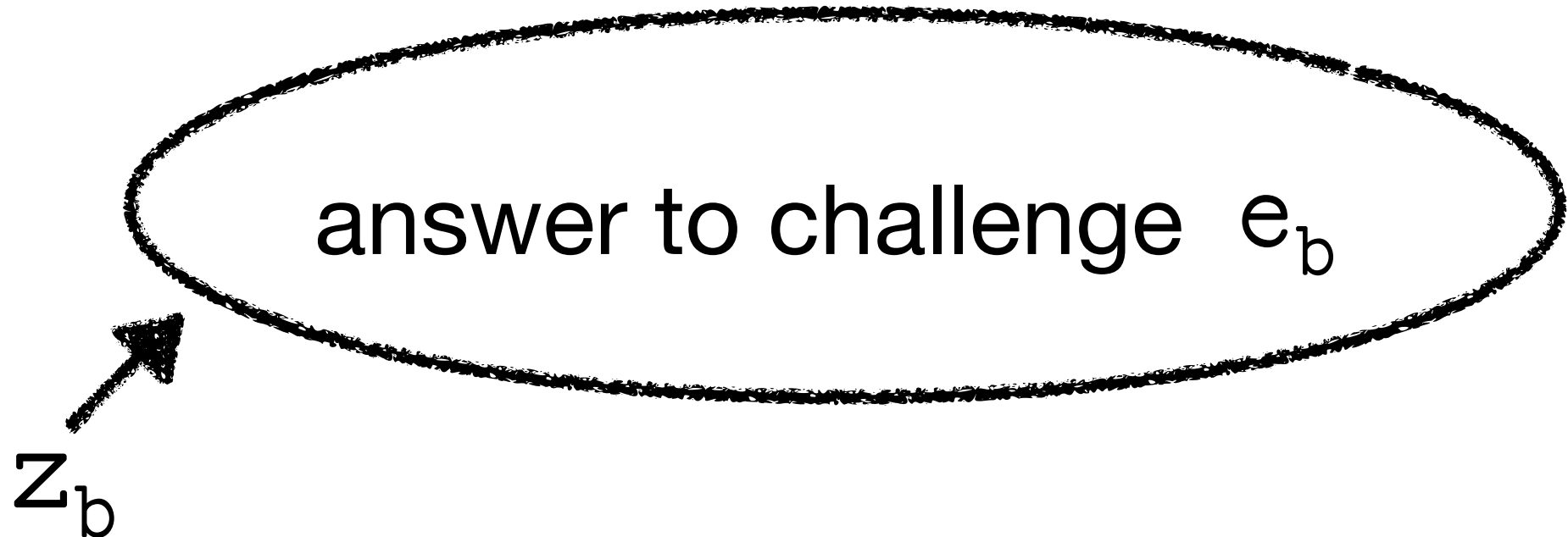
The verifier should not learn *which of the statements* the prover knows.

Sigma Protocol: OR Proof



Prover

$$e_b = s \oplus e_{1-b}$$



a_b using (x_b, w)
 a_{1-b} e_{1-b}
Chosen at random

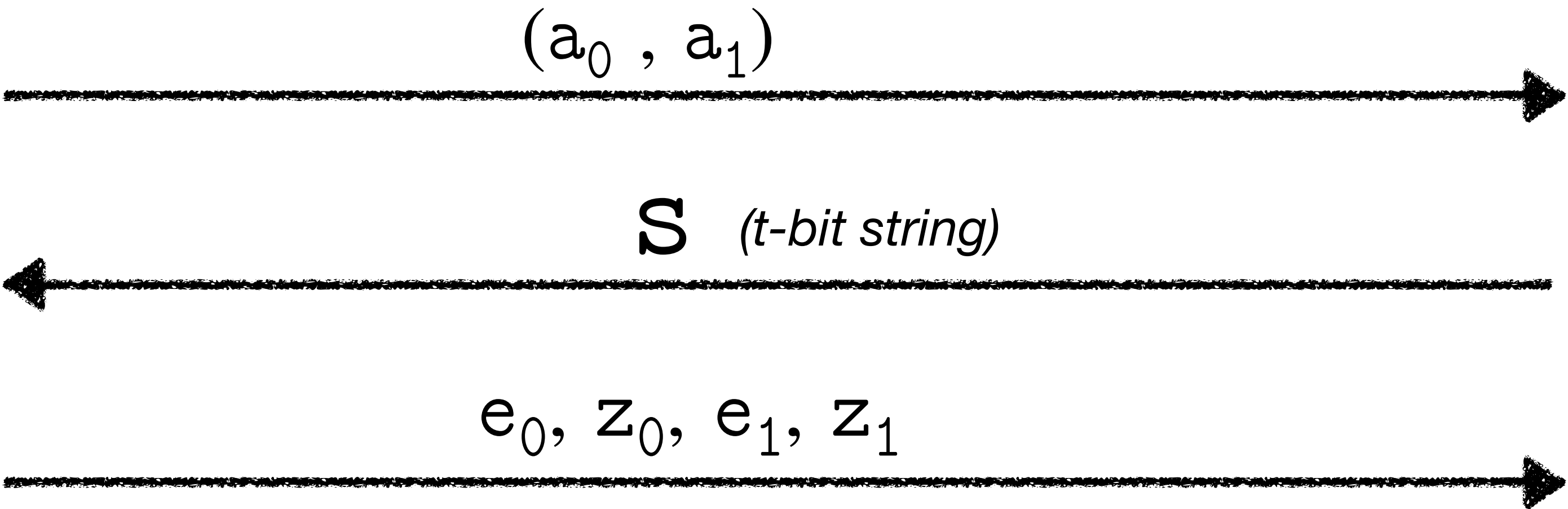


Verifier

$$e_0 \oplus e_1 = s$$

(a_0, e_0, z_0)
 (a_1, e_1, z_1)

Accepting Conversations in P



More Sigma Protocols

Prove that a relation R is a Diffie-Hellman Tuple.

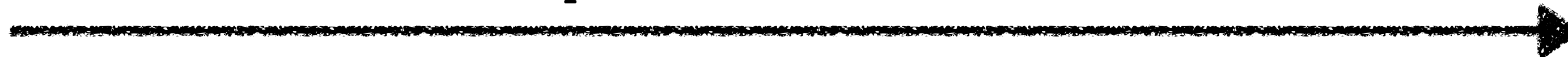
$$(g, h, u, v) \in R \quad \exists w, u = g^w \quad v = h^w$$

This is a proof of membership, but *not of knowledge*.

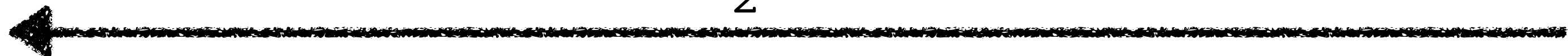


Prover

$$r \in \mathbb{Z}_q: \quad a = g^r \quad b = h^r$$



$$e \in \mathbb{Z}_{2^t}$$



$$z = r + e \cdot w \quad \text{mod } q$$



Verifier

checks: $g^z = au^e \text{ mod } p$
 $h^z = bv^e \text{ mod } p$

Sigma Protocols

If a Sigma-Protocol for R exists, then for any t , there exists a Sigma-Protocol for R with challenge length t .

Sigma Protocols

The properties of Sigma-Protocols are invariant under parallel composition. For instance repeating a Sigma-protocol for R twice in parallel produces a new Sigma-protocol for R with challenge length $2t$.

OR of many statements

Prove k-of-n statements $(x_1, \dots, x_n)$

A = Set of indices prover knows

B = Set of indices prover does not know

Field elements $(\alpha_1, \dots, \alpha_n)$ Polynomial f with free coefficient S

$f(\alpha_i)$ Share of S for P_i

For every $i \in B$ *prover generates* (a_i, e_i, z_i) using a simulator

For every $j \in A$ *prover generates* a_j as in the protocol

Prover sends $(a_1, \dots, a_n)$

OR of many statements

Prover sent: $(a_1, \dots, a_n)$

Verifier sends a random field element e

Prover finds a polynomial f of degree: $n - k$

Passes through all (α_i, e_i) $i \in B$ and $(0, e)$

The prover computes $e_j = f(\alpha_j)$ $j \in A$

Prover computes z_j based on (a_j, e_j)

Zero Knowledge from Sigma Protocols

We have seen that Sigma protocols are zero knowledge as long as the verifier is honest

What does honest mean?

Challenge, e , is independent of a

However, the verifiers are not necessarily honest!

So what can we do?

Commitment Schemes

Zero Knowledge from Sigma Protocols

The verifier first commits to its challenge e using a perfectly hiding commitment

Verifier sends a commitment: $c = \text{Com}(e ; r)$

Prover sends a message: a

Verifier sends (e , r)

Prover checks: $c = \text{Com}(e ; r)$

If yes: Sends a reply z

Verifier accepts based on: (x, a, e, z)

Zero Knowledge from Sigma Protocols

Simulate the transcript of the protocol when we have access only to the Verifier

Verifier commits

Send to verifier a message a' generated by the simulator, for a random e'

Receive Verifier's decommitment to e

Run the simulator again with e , rewind, V , and send a

Repeat until V decommits to e again

Conclude by sending z

Pedersen Commitments

Generator: g Order: q Commit to : x

The Receiver Chooses a random k and sends $h = g^k$

Sender Sends: $g^r \cdot h^x$

Why is this hiding?

For every x, y , there exists r, s , such that: $r + k \cdot x = s + k \cdot y \text{ modulo } q$

A commitment to x looks exactly like a commitment to y

Then why is it binding?

If we can find r, x, s, y

$$g^r \cdot h^x = g^s \cdot h^y$$

$$k = (r - s)/(y - x) \text{ module } q$$